第二节：URL与视图

一、URL与视图概念

二、定义URL

三、请求方法

四、页面重定向

一、URL与视图概念

当我们编写如下面的代码并运行：

app.py Python

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World!'

if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0',port=5000)
```

在浏览器进入 `https://127.0.0.1:5000` 后，可以看到前端显示：” Hello World! “，这就是 `hello_world` 函数返回的值。

二、定义URL

1. 定义无参URL

下面是无参URL的定义方法：

app.py Python

```python
import flask as Flask

app = Flask(__name__)

@app.route('/bolg')  # 这是无参路由
def bolg():
    return 'Hi,bolg!'

if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0',port=5000)
```

直接访问 `https://127.0.0.1:5000/bolg` 会在前端显示：” Hi,boig! “。

2. 定义有参URL

在URL中传递参数往往有2种方式：

- 定义为 `/bolg/<bolg_id>` ，那么访问时就是 `/bolg/123` 就可以将参数 `bolg_id=123` 传递给后端视图函数，视图函数再在前端返回数据，这是path传参。
- 定义为 `/bolg` ，那么访问时就是 `/bolg?bolg_id=123` ，也可以将参数 `bolg_id=123` 传给视图函数，这叫做query string传参。

2.1. path传参

app.py Python

```python
import flask as Flask

app = Flask(__name__)

@app.route('/blog/<int:blog_id>')  # 这是有参路由，当然这里可以是其他类型float，string等
def blog(blog_id):
    return f"获得的id为：{blog_id}"

if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0',port=5000)
```

我们可以看到参数定义为 `<类型:参数名>` ，如果设置了类型，在解析时会将参数转换为当前类型，倘若转换错误，就会报404。如果没有设置，就默认为字符串类型。

- `string` ：字符串类型
- `int` ：整数类型
- `float` ：浮点数
- `path` ：路径
- `uuid` ：UUID类型
- `any` ：备选值中的任意一个，比如： `@app.route("/bolg/<any(python,java):test>")` ，那么参数test就是 `python，java` 中的一个，否则就会出现404。

2.2. query syting传参

app.py Python

```python
import flask as Flask,request

app = Flask(__name__)

@app.route('/blog') # query 传参 使用 http://127.0.0.1:5000/blog?blog_id=666 访问id
def blog_detail():
    blod_id = request.args.get('blog_id')
    return f"blog的id为：{blod_id}"

@app.route('/page') # 可以实现多参数获取
def page_list():
    # 获取名为 page_num 的参数，如果没有获取到，默认为10
    # 可以通过参数type指定该page_num为整形，获取到的参数就会被强制转换为整形
    # 否则都是字符串类型
    page_num = request.args.get('page_num',10,type=int)
    page_size = request.args.get('page_size',5,type=int)
    return f"page_num:{page_num},page_size:{page_size}"

if __name__ == '__main__':
    app.run(debug=True,host='0.0.0.0',port=5000)
```

可以使用 `参数名 = request.args.get("参数名")` 形式获取到参数。其他可以看注释。

三、请求方法

在定义URL时，不同操作代表不同方法。从服务器获取数据使用 `get` 请求，提交数据到服务器就是 `post` 请求，修改服务器数据就是 `put` 请求，从服务器获取数据就是 `delete` 请求。代码如下：

app.py Python

```python
@app.route('/blog/add',methods=['POST'])
def blog_add():
    return "使用POST方法添加博客"
```

运行结果如下：

但如果添加 `GET` 方法：

app.py Python

```python
@app.route('/blog/add',methods=['POST''GET'])
def blog_add():
    return "使用POST方法添加博客"
```

运行结果如下：

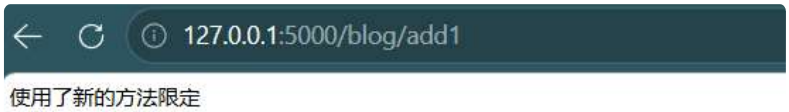
上面的代码显示了代码只能使用 `POST` 方法请求添加博客，如果使用其他方法的话，就会返回 `405 Method Not Allowed` 错误。当然，除了使用 `@app.route()` 装饰器，还有以下方法：

快捷路由装饰器	描述
app.get("/login")	等价于app.route("/login",methods=['GET'])
app.post("/login")	等价于app.route("/login",methods=["POST"])
app.put("/login")	等价于app.route("/login",methods=["PUT"])
app.delete("/login")	等价于app.route("/login",methods=["DELETE"])
app.patch("/login")	等价于app.route("/login",methods=["PATCH"])

代码及其结果如下：

app.py Python

```python
@app.get('/blog/add1')
def blog_add1():
    return "使用了新的方法限定"
```



四、页面重定向

页面和重定向，下文我们简称重定向，重定向在页面中体现的操作是，浏览器会从一个页面自动跳转到另外一个页面。比如用户访问了一个需要权限的页面，但是该用户当前并没有登陆，因此我们应该给他重定向到登陆页面。

重定向分为永久性重定向和暂时性重定向，以下是相关介绍：

永久性重定向：http的状态码是301，多用于旧网址被度弃，要转到一个新的网址确保用户正常的访问。最经典的就是京东网站案例，他在使用 `wwwjd.com` 域名之前有过许多其他域名，比如 `www.360buy.com`、`wwwijingdong.com` ，以前在这两个域名没有被废弃之前，当用户在浏览器中输入这两个域名的时候，会自动跳转到 `wwwjd.com` ，因为这两个域名以后要被度弃了，所以这种情况下应该使用永久性重定向。

暂时性重定向：http的状态码是302，表示页面的暂时性跳转，比如访问一个需要权限的网址，但是当前用户没有登陆，这时候就应该重定向到登陆页面，并且是暂时性的重定向。

在Flask中，重定向是通过 `flask.redirect(location,code=302)` 函数来实现的， `location` 表示需要重定到哪个URL， `code` 代表状态码，默认是302,也即暂时性重定向。下面我们用一个简单的案例来说明下这个函数的用法：

app.py Python

```python
from flask import Flask,request,redirect

app = Flask(__name__)

@app.route('/no_login')
def fuck():
    return 'fuck you'

@app.route('/login',methods=['POST','GET'])
def login():
    # 简单判断：传了name参数就是登录，没传就是没登陆
    name = request.args.get('name')

    if name is None:
        return redirect('/fuck')
    else:
        return f"你好{name}，欢迎登录！"

if __name__ = "__main__":
    app.run()
```

当我们访问 `https://127.0.0.1:5000/login` 时，由于没有携带 `name` ，所以会被重定向到 `/no_login` 路由下，前端显示 `fuck you` 的字样，但如果访问 `https://127.0.0.1:5000/login?name=wedeout` ，前端就会显示 `你好wedeout，欢迎登录！` 。